

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 5

Total Marks: 50

CASE STUDY

1. An embedded system has limited physical memory (64 MB) and runs several small applications. Each application consists of multiple modules, and memory usage must be minimized. The system should load only the required modules when necessary.

Tasks:

- a. Evaluate whether static linking or dynamic linking would be more suitable for this system.
 - b. Compare the memory and storage requirements of the two approaches.
 - c. Identify one disadvantage of your selected approach.
 - d. Justify your final choice based on the requirements of the embedded system.
2. A cloud server runs hundreds of processes simultaneously. Processes have different memory requirements and may start or terminate at any time. The system should minimize external fragmentation and efficiently utilize available physical memory.

Tasks:

- a. Evaluate contiguous memory allocation and paging for this system.
 - b. Determine which mechanism would provide better memory utilization.
 - c. Explain how the selected mechanism handles processes that require non-contiguous memory.
 - d. Justify your choice based on fragmentation, memory utilization, and process management.
3. A software application consists of several logically independent components such as:
 - Code

- Global data
- Stack
- Heap
- Shared libraries

The developers want memory protection between these components and want different access permissions for each logical section.

Tasks:

- Evaluate whether paging or segmentation is more suitable for this application.
 - Explain how the selected technique can provide protection and controlled access.
 - Identify one limitation of the selected technique.
 - Justify your choice based on logical organization, protection, and memory utilization.
4. A modern operating system must support:
- Large applications
 - Multiple processes
 - Efficient physical memory utilization
 - Protection between processes
 - Virtual memory
 - Programs larger than available physical memory

The system designers are considering paging, segmentation, and segmentation with paging.

Tasks:

- Compare the three memory management mechanisms for the given requirements.
 - Evaluate which mechanism provides the best combination of protection, flexibility, and memory utilization.
 - Explain how your selected mechanism handles logical memory and physical memory.
 - Justify why the other two mechanisms are comparatively less suitable.
5. Consider two systems:

System A:

A high-performance gaming computer with large RAM, where applications require fast memory access and frequently load large executable files.

System B:

A multi-user server running many independent processes with varying memory requirements.

The OS designers are considering contiguous allocation, paging, and segmentation.

Tasks:

- Evaluate the suitability of each memory management technique for System A and System B.

- b. Select the most appropriate mechanism for each system.
- c. Compare your choices in terms of:
 - a. Fragmentation
 - b. Memory utilization
 - c. Protection
 - d. Address translation overhead
 - e. Flexibility
- d. Provide a final recommendation for both systems with appropriate justification.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

08/2026

MONDAY

S. ROHINI

192424413

ESA0406 => OPERATING SYSTEMS

CO3_AT2

Q1) Static Linking vs Dynamic Linking:

case: The embedded system has only 64MB physical memory, runs several small applications, and should load only the required modules when necessary.

a) Suitable Linking Technique:

Dynamic linking is more suitable:

Feature	Static Linking	Dynamic Linking
Linking Time	Before Execution	During execution
Library	Entire required code included in executable	Modules loaded when required
Memory usage	Higher	Lower
Storage	Larger executable	Smaller executable.
Sharing	Less efficient	Libraries/modules can be shared

Since the system has limited 64MB memory and needs to load modules only when required, dynamic linking better satisfies the requirements.

b) Memory and Storage requirements:

Static Linking:

Higher memory and storage because library code is included in each program.

Dynamic Linking:

Lower memory and storage because modules are loaded when needed and can be shared.

c) Disadvantage of Dynamic Linking:

The main disadvantage is runtime overhead.

⇒ The required module must be located and loaded during execution, which can increase startup or execution time.

d) Justification:

Dynamic linking is preferred because it reduces memory usage, avoids duplicate modules and loads only required modules. Hence, it is best for the memory-limited embedded systems.

Q2) Contiguous Allocation vs Paging:

The cloud server runs hundreds of processes with varying memory requirements. Processes can start or terminate at any time, so the system must reduce external fragmentation and use physical memory efficiently.

a) Evaluation :

Contiguous Allocation	paging
process needs continuous memory	process can use non-contiguous memory
causes external Fragmentation	Eliminates external fragmentation
Difficult to manage varying process sizes.	Efficient for different process sizes.

b) Better memory utilization :

paging provides better memory utilization because a process does not need one continuous block of physical memory.

⇒ Non-Contiguous Memory :

paging divides a process into fixed size pages and physical memory into frames. pages can be placed in different free frames, allowing the process to occupy non-Contiguous memory.

d) Justification :

paging is better choice for the Cloud Server because

It :

⇒ Reduces external fragmentation.

⇒ Improves memory utilization

⇒ Supports processes with different memory required.

⇒ Allow processes to use non-Contiguous Memory.

Q3) paging vs Segmentation:

The Application Contains Code, Global Data, Stack Heap and Shared Libraries. Since these are logically independent components and require different access permissions, Segmentation is more suitable.

a) Suitable technique:

Segmentation divides the program into logical Segments such as:

=> Code

=> Stack

=> Data

=> Heap

=> Shared Libraries.

This matches the logical structure of the application better than paging.

b) protection and Controlled Access:

Each segment can have its own base address, limit, and access permissions.

For example:

=> code -> Read / Execute

=> Data -> Read / write

=> Stack -> Read / write

=> Shared Libraries -> Controlled Access

The OS checks these permissions before allowing access, providing memory protection between different components.

c) Limitation:

A major limitation of Segmentation is external fragmentation because segments have variable sizes. Over time, free memory may become divided into small unused spaces.

d) Justification:

Segmentation is preferred because it provides:

=> Logical organization of program components.

=> Different access permissions for each segment.

=> Better memory protection.

=> Flexible Allocation based on segment size.

Q4) Paging vs Segmentation vs Segmentation with paging:

The modern OS must support large applications, multiple processes, efficient physical memory use, protection, virtual memory use, protection, virtual memory, and programs larger than physical memory.

a) Comparison:

Mechanism	Main Feature	Limitation
Paging	Divides memory into fixed-size pages and frames	Less logical organization.
Segmentation	Divides programs into logical segments	Can cause external fragmentation.

Segmentation with paging	Combines logical segments with paging	More complex address translation
--------------------------	---------------------------------------	----------------------------------

b) Best Mechanism:

Segmentation with paging provides the best combination of protection, flexibility and memory utilization.

⇒ Segmentation provides logical organization and protection.

⇒ paging reduces external fragmentation.

c) Logical and Physical Memory:

In Segmentation with paging, the program is first divided into logical segments. Each segment is further divided into pages. These pages are placed into available physical memory frames.

d) Why the other two are less suitable:

⇒ paging: provides good memory utilization but does not naturally represent logical program sections.

⇒ Segmentation: provides logical organization and protection but can suffer from external.

⇒ Segmentation with paging: Combines the advantages of both and is therefore more suitable for the given requirements.

Memory Management Techniques for System A and B.

The case compares a high-performance gaming Computer (System A) with a multi-user server (System B). The OS designers consider Contiguous allocation, paging, and Segmentation.

a) Suitability of each technique:

Technique	System - A	System - B
Contiguous Allocation	Suitable for fast access, but large programs need continuous memory.	Less suitable due to fragmentation.
Paging	Suitable for large applications and flexible memory Allocation.	Highly suitable for many processes with different memory requirements.
Segmentation	useful for logical organization and protection.	useful for protection, but external fragmentation can occur.

b) Most Appropriate Mechanism:

System A $\Rightarrow$ Contiguous Allocation.

$\Rightarrow$ Large RAM is available

$\Rightarrow$ Fast memory access is important

$\Rightarrow$ Large executable files are frequently loaded.

System B is paging

- ⇒ Many independent processes run simultaneously.
- ⇒ processes have varying memory requirements.
- ⇒ paging allows non-contiguous memory Allocation.

c) Comparison:

Factor	System A	System B
Fragmentation	can cause external Fragmentation.	Minimizes external Fragmentation.
Memory utilization	Good with large RAM	Better with paging
protection	Limited	Better with paging
Translational Overhead	Low	Some page-table Overhead
Flexibility	lower	Higher

d) Recommendation:

System A ⇒ Contiguous Allocation is recommended

because the system has large RAM and requires fast memory access.

System B ⇒ paging is recommended because it provides better memory utilization, flexibility and reduced external fragmentation for many processes with varying memory requirements.